

OpenAI Agents SDK in Python

Introduction, use cases, and examples for Business Analytics students

Armando Vieira - Tartu University, 2026

April 2026

Learning goals

- Understand what the OpenAI Agents SDK is and where it fits in an analytics workflow.
- Distinguish between a plain LLM call and an **agentic** system.
- See how Python tools, structured workflows, and multi-agent orchestration can support business analysis.
- Read and adapt simple code examples in Python.
- Evaluate when to use agents, and when not to use them.

Why business analytics students should care

Traditional analytics work

- Query data
- Clean and transform it
- Compute KPIs
- Write a short interpretation
- Recommend an action

What agents add

- Natural-language interface
- Automatic tool usage
- Multi-step reasoning across tasks
- Routing to specialists
- Faster decision-support workflows

Idea: instead of writing one large fixed script, you define an agent that can decide *which step to do next* and *which tool to call*.

What is the OpenAI Agents SDK?

Definition

The OpenAI Agents SDK is a Python framework for building applications where an **agent** uses an LLM together with instructions, tools, guardrails, sessions, and handoffs.

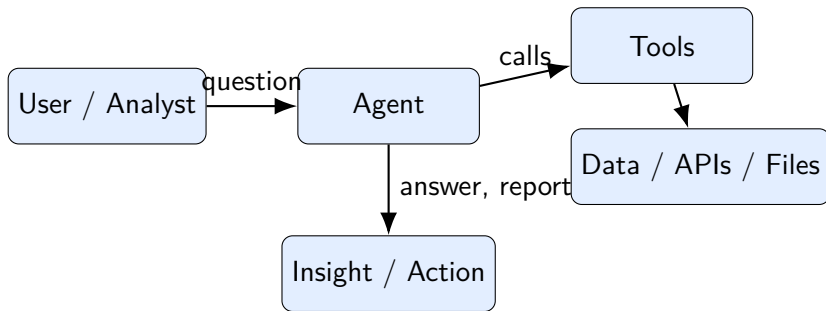
- An **agent** is more than a chatbot.
- It can call tools, manage multi-turn state, stream progress, and delegate to other agents.
- The SDK provides orchestration so the developer does not need to manually manage every model-tool loop.

Mental model: $LLM + role + tools + workflow\ logic = agent$.

Core concepts

- ① **Agent** - a role with instructions and optional model settings.
- ② **Runner** - executes the agent and coordinates the loop.
- ③ **Tools** - Python functions or hosted tools the agent can call.
- ④ **Handoffs** - transfer work to another specialized agent.
- ⑤ **Guardrails** - checks on input or output.
- ⑥ **Sessions / state** - preserve context across turns.
- ⑦ **Streaming and tracing** - observe progress and debug behavior.

Simple architecture



Examples of tools in analytics: SQL query, KPI calculator, forecasting function, anomaly detector, dashboard exporter.

Why business analytics students should care

- A standard model call can answer questions, but an **agent** can also decide when to use tools, when to hand work to another specialist, and how to complete a multi-step task.
- The OpenAI Agents SDK provides this orchestration layer in Python: Agent + Runner + tools + sessions + handoffs.
- For analytics work, that means one workflow can:
 - read a KPI request,
 - fetch data from a function or API,
 - compute metrics,
 - explain results in business language.
- This is useful for dashboards, automated reporting, anomaly investigation, and decision support.

What the SDK is

Core idea

The SDK helps developers build agents that plan, call tools, collaborate across specialists, and maintain enough state to complete multi-step work.

- Official quickstart uses Agent and Runner as the main building blocks.
- The SDK uses the **Responses API by default** for OpenAI models.
- It also supports important orchestration features such as:
 - tools,
 - sessions,
 - handoffs,
 - guardrails,
 - streaming,
 - tracing.

Main SDK components

Component	Role in an analytics workflow
Agent	Defines the specialist: e.g. KPI explainer, forecasting assistant, marketing analyst
Runner	Executes the workflow loop until final output is produced
Tools	Let the agent call Python functions, APIs, code, or external systems
Sessions	Preserve conversation history across multiple turns
Handoffs	Allow one specialist agent to pass work to another
Guardrails	Add constraints and validation around behavior or outputs

Mental model: from prompt to agent workflow

User request → Agent reasoning → Tool use → Optional handoff → Final answer

- A normal prompt is often one-shot.
- An agent run is iterative.
- According to the SDK docs, the loop is roughly:
 - ① invoke the agent,
 - ② if final output is ready, stop,
 - ③ else run a handoff or tool calls,
 - ④ run again until completion.

Installation and first setup

Create and activate a virtual environment

```
python -m venv .venv
```

```
source .venv/bin/activate
```

Install the SDK

```
pip install openai-agents
```

Set your API key

```
export OPENAI_API_KEY="sk-..."
```

- Keep projects isolated with a virtual environment.
- In production, store the API key in environment variables or a secrets manager.
- For classroom demos, keep the setup minimal before adding tools and data.

Minimal example: one agent

```
import asyncio
from agents import Agent, Runner

agent = Agent(
    name="Analytics Tutor",
    instructions=(
        "Explain business analytics concepts clearly, "
        "briefly, and with examples."
    ),
)

async def main():
    result = await Runner.run(
        agent,
        "Explain the difference between revenue and profit."
    )
    print(result.final_output)
```

What makes this agentic?

A simple example above still looks like a chat interface. It becomes more **agentic** when the system can:

- decide to call a tool,
- inspect returned data,
- continue reasoning after the tool result,
- route the task to another agent,
- preserve context across turns,
- expose progress with streaming and traces.

In analytics, this matters because real tasks often require **data access + calculation + interpretation + recommendation**, not just text generation.

Function tools: turning Python functions into tools

```
import asyncio
from agents import Agent, Runner, function_tool

@function_tool
def quarterly_sales() -> dict:
    """Return sample quarterly sales in thousands of euros."""
    return {"Q1": 120, "Q2": 145, "Q3": 160, "Q4": 210}

agent = Agent(
    name="Sales Analyst",
    instructions=(
        "Answer questions about sales performance. "
        "Use quarterly_sales when data is needed."
    ),
    tools=[quarterly_sales],
)

async def main():
```

Business example 1: KPI explanation agent

```
import asyncio
from agents import Agent, Runner, function_tool

@function_tool
def margin_metrics(revenue: float, cost: float) -> dict:
    """Compute gross profit and gross margin."""
    gross_profit = revenue - cost
    gross_margin = gross_profit / revenue
    return {
        "gross_profit": round(gross_profit, 2),
        "gross_margin": round(gross_margin, 4)
    }

agent = Agent(
    name="KPI Coach",
    instructions=(
        "Use the tool when needed. Explain both the formula "
        "and the managerial meaning of the result."
```

Business example 2: anomaly triage

```
from agents import Agent, function_tool

@function_tool
def detect_anomaly(current: float, expected: float) -> dict:
    """Compare observed and expected value."""
    gap = current - expected
    pct_gap = gap / expected
    return {
        "gap": round(gap, 2),
        "pct_gap": round(pct_gap, 4),
        "is_material": abs(pct_gap) >= 0.10,
    }

anomaly_agent = Agent(
    name="Anomaly Analyst",
    instructions=(
        "Flag material deviations and explain likely business causes "
        "such as seasonality, stockouts, pricing, or campaign effects."
    ),
    tools=[detect_anomaly],
)
```

Typical analytics question: *"Daily sales were 42k, but forecast was 50k. Is this a meaningful deviation?"*

Use cases in business analytics

Operational

- Sales monitoring
- Inventory alerts
- Campaign performance summaries
- Customer-support triage

Strategic

- Pricing decision support
- Executive reporting
- Scenario comparison
- Risk flagging

Analytical

- KPI calculation and explanation
- Segmentation support
- Forecast interpretation
- Root-cause analysis

Educational

- Analytics tutor
- Coding helper for notebooks
- SQL explanation assistant
- Case-study simulator

A practical analytics workflow using agents

- ① User asks a business question in plain language.
- ② Agent identifies what information is missing.
- ③ Agent calls one or more tools.
- ④ Tool returns structured data.
- ⑤ Agent interprets results in business language.
- ⑥ Output is delivered as a short answer, report paragraph, alert, or recommendation.

Example:

- Question: “Why is Region A underperforming this week?”
- Possible tool chain: sales query → forecast comparison → stockout check → summary.

Structured outputs for downstream systems

Often we need machine-readable outputs for dashboards, pipelines, or alerts.

```
from pydantic import BaseModel
from agents import Agent

class AlertDecision(BaseModel):
    severity: str
    explanation: str
    recommended_action: str

agent = Agent(
    name="Alert Generator",
    instructions="Classify anomalies and recommend a next action.",
    output_type=AlertDecision,
)
```

Why useful? Easier validation, storage, and integration into BI workflows.

Multi-agent design: specialists instead of one giant prompt

```
from agents import Agent

forecast_agent = Agent(
    name="Forecast Specialist",
    instructions="Analyze forecasts and explain model-related issues."
)

marketing_agent = Agent(
    name="Marketing Analyst",
    instructions="Explain campaign, channel, and conversion effects."
)

manager_agent = Agent(
    name="Business Manager",
    instructions="Route questions to the right specialist.",
    handoffs=[forecast_agent, marketing_agent],
)
```

Design principle: concrete responsibilities. This improves clarity, reuse, and maintainability.

Use case 1: automated management reporting

- A manager asks: *"What happened this week?"*
- The agent can:
 - ① fetch latest KPIs,
 - ② compare them to targets,
 - ③ detect exceptions,
 - ④ write a concise narrative summary.
- This reduces manual reporting time and makes explanations more consistent.
- Good fit for revenue reports, campaign updates, supply-chain summaries, or board prep notes.

Use case 2: anomaly detection assistant

- Business question: *“Why did sales fall in March?”*
- The agent can coordinate several steps:
 - inspect region-level sales,
 - check marketing spend,
 - compare product categories,
 - highlight likely drivers.
- This is especially useful when non-technical users want explanations, not just raw numbers.

Use case 3: customer analytics and churn

- An agent can support segmentation and retention analysis.
- Example tasks:
 - identify high-risk customer groups,
 - summarize churn patterns,
 - generate retention recommendations,
 - draft a short executive memo.
- This is a strong example for business analytics students because it combines metrics, interpretation, and action.

When multi-agent systems help

Use multiple agents when the task naturally splits into **different expertise areas**.

Need	Possible specialist agent
Forecast diagnostics	Time-series / forecasting specialist
Customer churn interpretation	Retention specialist
Marketing attribution	Campaign specialist
Finance commentary	FP&A specialist
Data-quality issues	Validation / QA specialist

Avoid multi-agent complexity if one well-designed agent and a few tools are enough.

Guardrails and business risk

In business settings, a polished answer is not enough. It must also be **safe, relevant, and checkable**.

Why guardrails matter

- Prevent off-topic use
- Limit risky actions
- Check output format
- Support compliance and governance

Examples

- Require human approval before sending a client-facing email.
- Reject answers that reveal confidential data.
- Enforce a structured JSON schema for alerts.

Streaming, sessions, and tracing

- **Streaming:** show partial progress while the run is happening.
- **Sessions:** preserve conversation history across turns.
- **Tracing:** inspect runs for debugging and evaluation.

Why these matter in analytics apps

- Users trust systems more when they can see progress.
- Analysts often ask follow-up questions; sessions preserve context.
- Traces help developers understand which tool was called and why a conclusion was produced.

Strengths and limitations

Strengths

- Natural-language interface
- Flexible workflows
- Tool calling and orchestration
- Good fit for decision support
- Can reduce repetitive reporting work

Limitations

- Needs careful tool design
- Can still make incorrect inferences
- Requires evaluation and monitoring
- Not a substitute for data governance
- Adds architectural complexity

Limitations and risks

- Agents can still hallucinate or misinterpret ambiguous requests.
- Weak tools produce weak analytics.
- Poor data quality leads to poor recommendations.
- More autonomy means stronger need for validation, monitoring, and guardrails.

Rule of thumb

Use agents to accelerate analysis and communication, but keep humans responsible for high-stakes business decisions.

Best practices for student projects

- ① Start with one agent and one or two tools.
- ② Keep tools deterministic and easy to test.
- ③ Separate **data retrieval**, **calculation**, and **explanation**.
- ④ Prefer structured outputs when the result will feed another system.
- ⑤ Log runs and inspect traces.
- ⑥ Add human review for high-stakes actions.
- ⑦ Evaluate using realistic business cases, not only toy prompts.

When not to use an agent

An agent is not always the best solution.

- If the task is a fixed formula, plain Python may be simpler.
- If the workflow is fully deterministic, a normal pipeline may be easier to audit.
- If the task is highly regulated, you may need stronger approval and validation layers.
- If the answer depends on precise numerical reproducibility, compute first and let the agent explain later.

Rule of thumb: use agents for *judgment + orchestration + explanation*, not for replacing all conventional software engineering.

Mini case study: retail performance assistant

Scenario

A retailer wants an assistant that explains sudden weekly underperformance.

Possible tool stack

- sales query tool
- forecast comparison tool
- stock availability tool
- campaign calendar tool

Expected output

- 1 Detect whether the deviation is material.
- 2 Identify likely drivers.
- 3 Summarize findings in plain business English.
- 4 Recommend next steps: restock, review campaign pacing, or investigate pricing.

Key takeaways

- The OpenAI Agents SDK helps developers build systems that combine **language understanding** with **actions and workflow logic**.
- For business analytics, agents are especially useful when a task combines **data access, calculation, interpretation, and recommendation**.
- Start small: one agent, a small toolset, and a clear evaluation plan.
- The strongest designs treat agents as part of a wider analytics system, not as magic replacements for models, code, or judgment.

Official references

- OpenAI Agents SDK docs: <https://openai.github.io/openai-agents-python/>
- Quickstart: <https://openai.github.io/openai-agents-python/quickstart/>
- Tools: <https://openai.github.io/openai-agents-python/tools/>
- Agents: <https://openai.github.io/openai-agents-python/agents/>
- Streaming: <https://openai.github.io/openai-agents-python/streaming/>